

Technical Document #245: Software Engineering - Comprehensive Overview

Technical Document #245: Software Engineering - Comprehensive Overview

Technical debt represents the cost of choosing easy solutions over better ones. It accumulates over time and slows development velocity.

Test-driven development (TDD) writes tests before writing code. It improves code quality and design through small, incremental changes.

Domain-driven design (DDD) models software around business domains. It improves communication between technical and business stakeholders.

API design defines how software components communicate. RESTful APIs and GraphQL are popular API design approaches.

Continuous integration (CI) automatically builds and tests code changes. It catches integration issues early in the development process.

Refactoring improves code structure without changing behavior. It makes code easier to understand, maintain, and extend.

Robotics combines mechanical engineering, electrical engineering, and computer science. Modern robots can perform complex tasks in unstructured environments.

Cybersecurity threats continue to evolve in complexity and scale. Organizations must invest in robust security measures to protect sensitive data.

Artificial intelligence continues to reshape industries from healthcare to finance. Machine learning models can now perform tasks that were once thought to require human intelligence.

Key Takeaways:

- Domain-driven design (DDD) models software around business domains.
- Technical debt represents the cost of choosing easy solutions over better ones.
- Code review examines code changes before merging.